day-02

Consultancy Platform Backend — Day 2 Progress Report

Project Overview

Started building a production-ready consultancy platform backend using:

- FastAPI
- PostgreSQL
- SQLAlchemy ORM
- Alembic
- Pydantic Settings
- Async Architecture

The backend follows scalable SaaS backend architecture principles with layered structure and production-ready database management.

Completed Today

1. Backend Project Initialization

Initialized FastAPI backend project structure.

Created base folders:

```
backend/
├── app/
├── alembic/
├── requirements.txt
├── .env
├── alembic.ini
```

---

2. Installed Core Dependencies

Installed:

```
fastapi
uvicorn
sqlalchemy
asyncpg
alembic
pydantic
pydantic-settings
python-dotenv
psycopg2-binary
```

Also installed PostgreSQL locally using Homebrew.

---

3. Environment Configuration

Created `.env` file.

Current environment variables:

```
APP_NAME=Consultancy Platform
APP_ENV=development

DB_HOST=localhost
DB_PORT=5432
DB_NAME=consultancy_platform
DB_USER=arimardanpandey
DB_PASSWORD=

SECRET_KEY=supersecretkey
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30
```

---

4. PostgreSQL Setup

Successfully:

- Installed PostgreSQL 17
- Initialized PostgreSQL database cluster
- Started PostgreSQL service
- Connected using `psql`
- Created database:

```
CREATE DATABASE consultancy_platform;
```

Verified database creation successfully.

---

5. FastAPI Application Running

Successfully started FastAPI server using:

```
uvicorn app.main:app --reload
```

Application running successfully on:

```
http://127.0.0.1:8000
```

---

6. Base Database Architecture

Created reusable database foundation.

Created:

`app/db/base.py`

Contains SQLAlchemy Declarative Base.

`app/db/mixins.py`

Created reusable mixins:

- UUIDMixin
- TimestampMixin

Used for:

- automatic UUID primary keys
- created_at timestamps
- updated_at timestamps

---

7. User Role Enum

Created:

```
app/core/constants.py
```

Added production role enum:

```python
class UserRole(str, Enum):
    ADMIN = "ADMIN"
    CONSULTANT = "CONSULTANT"
    CLIENT = "CLIENT"
    SUPER_ADMIN = "SUPER_ADMIN"
```

---

8. User Model Created

Created production-ready `User` model.

Fields

```
id
first_name
last_name
email
phone
password_hash
```

```
role
is_active
is_verified
created_at
updated_at
```

Features:

- UUID primary key
- unique email
- unique phone
- PostgreSQL ENUM role
- timestamps
- indexes

---

9. Alembic Setup

Initialized Alembic successfully.

Configured:

- `alembic.ini`
- `alembic/env.py`
- `Base.metadata`
- model imports

Configured Alembic to use `.env` database configuration dynamically.

---

10. First Migration Generated

Successfully generated migration:

```
alembic revision --autogenerate -m "create users table"
```

Migration detected:

- users table
- email index

---

11. Migration Applied Successfully

Executed:

```
alembic upgrade head
```

Successfully created tables:

```
users
alembic_version
```

Verified using PostgreSQL `\dt`.

---

12. Verified Database Schema

Verified final schema using:

```
\d users
```

Confirmed:

- primary key
- UUID
- indexes
- unique constraints
- enum role
- timestamps

All working correctly.

Current Architecture

```
FastAPI
↓
SQLAlchemy ORM
↓
Alembic
↓
PostgreSQL
```

Architecture style:

```
Router
↓
Service
↓
Repository
↓
Database
```

Planned production SaaS architecture.

Current Folder Structure

```
backend/
├── app/
│   ├── api/
│   ├── core/
│   │   ├── config.py
│   │   └── constants.py
│   │
│   ├── db/
│   │   ├── base.py
│   │   └── mixins.py
│   │
│   ├── models/
│   │   ├── __init__.py
│   │   └── user.py
│   │
│   └── main.py
│
├── alembic/
```

```
├── alembic.ini
├── .env
```

Major Problems Solved Today

PostgreSQL Issues

Resolved:

- broken PostgreSQL installation
- missing postgres.bki
- brew service startup failure
- database cluster initialization
- PostgreSQL role mismatch
- Alembic DB connection issues

Important Decisions Taken

Database

- PostgreSQL selected
- UUID primary keys selected
- Alembic for migrations
- SQLAlchemy ORM

Architecture

- Production layered architecture
- Repository-Service pattern
- Environment-based config

- Reusable mixins

Next Development Phase

Authentication System

Next tasks:

1. Password Hashing
2. JWT Authentication
3. Register API
4. Login API
5. Current User API
6. Authentication Middleware
7. RBAC System

Planned Next Modules

```
Authentication
↓
RBAC
↓
Consultants Module
↓
Consultation Booking
↓
Payments
↓
Notifications
↓
Admin Panel APIs
```

Current Project Status

Completed

- Backend foundation
- Database setup
- ORM setup
- Migration system
- First production model

In Progress

- Authentication System

Not Started

- Business modules
- Booking system
- Payments
- Notifications
- File uploads
- Admin dashboard
- Consultant system

Notes For Next AI Agent

The project foundation is stable now.

Main focus tomorrow should be:

- Authentication module
- JWT token system
- Password hashing
- Register/Login APIs

- Dependency injection
- Repository-Service architecture

Avoid changing:

- database setup
- alembic setup
- ORM base structure
- mixin structure

Those are working correctly now.